REAL-TIME DATA ANALYTICS PLATFORM

Using Google Cloud Platform (GCP)

Complete Beginner's Implementation Guide

This guide walks you through every single step — from creating a Google account to seeing live data on a dashboard. No prior cloud experience needed!

What Are We Building?

We are building a system that:

- Generates real-time data (like fake sales or sensor readings)
- Sends that data to Google Cloud every second
- Processes and stores that data automatically
- Shows it on a live dashboard that updates in real time

> Think of it like this: Imagine a shop that records every sale, sends it to the cloud instantly, and you can see total sales updating live on a screen. That is exactly what we are building!

The Architecture (Big Picture)

Data flows through 4 stages:

Stage	GCP Service	What It Does
1. Ingest	Pub/Sub	Receives incoming data like a mailbox
2. Process	Dataflow	Cleans and transforms the data
3. Store	BigQuery	Saves data in a searchable table
4. Visualize	Looker Studio	Shows live charts and dashboards

Phase 1: Set Up Your Google Cloud Account

Time needed: 15-20 minutes | Cost: FREE (Google gives $300 in free credits)

Step 1.1 — Create a Google Cloud Account

1. Open your browser and go to: console.cloud.google.com
2. Sign in with your Google account (Gmail). If you do not have one, create it first.
3. You will be asked to start a free trial. Click 'Start Free Trial'.
4. Enter your name, country, and a credit/debit card number.

IMPORTANT: Google will NOT charge your card. It is only to verify you are a real person. You get $300 free credits which is more than enough for this entire project.

5. Click 'Start my free trial'. You are now inside the Google Cloud Console!

Step 1.2 — Create a New Project

A 'project' is like a folder for all your work. We will create one for this analytics platform.

6. In the top bar, you will see a dropdown (it may say 'My First Project' or 'Select a project').
7. Click that dropdown. A popup will appear.
8. Click 'NEW PROJECT' in the top right of the popup.
9. In the 'Project name' box, type: realtime-analytics-demo
10. Leave everything else as default. Click 'CREATE'.
11. Wait 10-15 seconds. Then click the notification bell to see 'Project created'.
12. Click 'SELECT PROJECT' in that notification.

You should now see 'realtime-analytics-demo' in the top bar. This means your project is active.

Step 1.3 — Enable Required APIs

GCP services need to be 'turned on' before you can use them. We need to enable 3 APIs.

Enable Pub/Sub API

13. In the search bar at the top, type: Pub/Sub API
14. Click on the first result 'Cloud Pub/Sub API'.
15. Click the blue 'ENABLE' button. Wait 30 seconds.

Enable Dataflow API

16. Search: Dataflow API in the top search bar.
17. Click 'Dataflow API'. Click 'ENABLE'. Wait 30 seconds.

Enable BigQuery API

18. BigQuery is usually already enabled. Search: BigQuery API
19. If it shows 'ENABLE', click it. If it shows 'MANAGE', it is already enabled.

Phase 2: Set Up BigQuery (Our Database)

> BigQuery is where all our data will be saved. Think of it like an Excel sheet but it can store billions of rows and lets you search them instantly.

Step 2.1 — Create a Dataset

A dataset in BigQuery is like a folder that holds your tables.

20. In the search bar, type BigQuery and click the first result.
21. You will see BigQuery Studio open. On the left panel, find your project name.
22. Click the 3 dots (...) next to your project name.
23. Click 'Create dataset'.
24. In 'Dataset ID', type: analytics_data
25. In 'Data location', choose: us-central1 (Iowa)
26. Leave everything else as default. Click 'CREATE DATASET'.

Step 2.2 — Create a Table

Now we create the actual table where our data rows will be stored.

27. Click the 3 dots next to analytics_data (the dataset you just made).
28. Click 'Create table'.
29. Under 'Destination', in 'Table' box, type: events
30. Under 'Schema', click 'Add field' and add these fields one by one:

Field Name	Type	Description
event_id	STRING	Unique ID for each event
user_id	INTEGER	Which user triggered the event
event_type	STRING	Type: page_view, click, purchase
product	STRING	Product name
amount	FLOAT	Amount in dollars
timestamp	TIMESTAMP	When it happened
city	STRING	User location

31. After adding all 7 fields, click 'CREATE TABLE'.

> Your table is now ready! It is empty for now but we will fill it with live data soon.

Phase 3: Set Up Pub/Sub (Message Queue)

> Pub/Sub is like a post office. Your Python script sends messages (data), Pub/Sub holds them, and Dataflow picks them up to process.

Step 3.1 — Create a Topic

32. In the search bar, type Pub/Sub and open it.
33. Click 'CREATE TOPIC'.
34. In 'Topic ID', type: realtime-events
35. Check 'Add a default subscription' (this auto-creates a subscriber).
36. Click 'CREATE'. You will see your new topic in the list.

Step 3.2 — Note Your Topic Path

After creation, click on realtime-events. You will see details including:

```
Topic name: projects/realtime-analytics-demo/topics/realtime-events

Save this path — you will need it in your Python code later!
```

Phase 4: Set Up Your Computer (Local Environment)

We need to install Python and some tools on your computer to send data to GCP.

Step 4.1 — Install Python

37. Go to python.org/downloads
38. Download Python 3.10 or higher.
39. During installation, CHECK the box that says 'Add Python to PATH'.
40. Click Install Now.
41. After installation, open a terminal (Command Prompt on Windows, Terminal on Mac).
42. Type the command below to verify Python is installed:

```
python --version
# Should show: Python 3.10.x or higher
```

Step 4.2 — Install Required Libraries

Open your terminal and run these commands one by one:

```
# Install Google Cloud libraries
pip install google-cloud-pubsub
pip install google-cloud-bigquery
pip install apache-beam[gcp]

# Wait for each to finish before running the next one
# Each may take 1-3 minutes to install
```

Step 4.3 — Install Google Cloud SDK

The Google Cloud SDK lets you run GCP commands from your terminal.

43. Go to: cloud.google.com/sdk/docs/install
44. Download the installer for your operating system (Windows/Mac/Linux).
45. Run the installer. Keep all default options.
46. When installation finishes, it will open a terminal and run gcloud init automatically.
47. A browser window opens — sign in with your Google account.
48. Select your project: realtime-analytics-demo
49. Back in terminal, verify it works:

```
gcloud --version

# Should show: Google Cloud SDK 450.x.x or similar

gcloud config get-value project
# Should show: realtime-analytics-demo
```

Step 4.4 — Authenticate Your Application

This step lets your Python scripts talk to GCP as you.

```
# Run this command in your terminal
gcloud auth application-default login

# A browser window will open
# Sign in with your Google account
# Click 'Allow'
# Terminal will show: Credentials saved to file: ~/.config/gcloud/...
```

After this step, any Python code on your computer can access your GCP project without needing passwords in the code!

Phase 5: Write the Data Producer (Python Script)

This script pretends to be a live website generating events: page views, purchases, clicks — and sends them to Pub/Sub every second.

Step 5.1 — Create Project Folder

```
# In your terminal, create a folder for your project
mkdir realtime-analytics
cd realtime-analytics

# Now create the Python file
# Windows: type nul > producer.py
# Mac/Linux: touch producer.py
```

Step 5.2 — Write producer.py

Open producer.py in any text editor (Notepad, VS Code, etc.) and paste this code:

```
# producer.py  —  Sends fake real-time events to Pub/Sub

import json
import time
import random
import uuid
from datetime import datetime
from google.cloud import pubsub_v1

# ─── CONFIGURATION ──────────────────────────────────────────
PROJECT_ID = 'realtime-analytics-demo'  # Your GCP project name
TOPIC_ID   = 'realtime-events'           # The topic we created
# ────────────────────────────────────────────────────────────

# Sample data to make realistic fake events
PRODUCTS = ['Laptop', 'Phone', 'Tablet', 'Headphones', 'Keyboard']
CITIES   = ['Mumbai', 'Delhi', 'Bangalore', 'Chennai', 'Hyderabad']
EVENTS   = ['page_view', 'click', 'add_to_cart', 'purchase']

# Create publisher client
publisher  = pubsub_v1.PublisherClient()
topic_path = publisher.topic_path(PROJECT_ID, TOPIC_ID)

def generate_event():
    """Creates one fake event with random data"""
    event_type = random.choice(EVENTS)
    amount = round(random.uniform(10, 2000), 2) if event_type == 'purchase' else
0.0
    return {
        'event_id'  : str(uuid.uuid4()),
        'user_id'   : random.randint(1, 500),
        'event_type': event_type,
        'product'   : random.choice(PRODUCTS),
```

```
        'amount'    : amount,
        'timestamp' : datetime.utcnow().isoformat() + 'Z',
        'city'      : random.choice(CITIES)
    }

def publish_event(event):
    """Sends one event to Pub/Sub"""
    data = json.dumps(event).encode('utf-8')  # Convert to bytes
    future = publisher.publish(topic_path, data)
    return future.result()  # Wait until published

# ─── MAIN LOOP ───
print(f'Starting data producer...')
print(f'Publishing to: {topic_path}')
print('Press Ctrl+C to stop')
print('-' * 40)

count = 0
try:
    while True:                          # Run forever
        event = generate_event()          # Make a fake event
        msg_id = publish_event(event)     # Send to Pub/Sub
        count += 1
        print(f'[{count}] Sent: {event["event_type"]} - {event["product"]} -
${event["amount"]}')
        time.sleep(1)                     # Wait 1 second

except KeyboardInterrupt:
    print(f'\nStopped. Total events sent: {count}')
```

What this script does: Every 1 second it makes a random event (like a user viewing a page or making a purchase), converts it to JSON, and sends it to Pub/Sub.

Phase 6: Write the Dataflow Pipeline (Data Processor)

The Dataflow pipeline reads messages from Pub/Sub, parses the JSON, and writes each row into BigQuery automatically.

Step 6.1 — Create pipeline.py

In your realtime-analytics folder, create a new file called pipeline.py and paste this:

```
# pipeline.py  —  Reads from Pub/Sub, writes to BigQuery

import json
import logging
import apache_beam as beam
from apache_beam.options.pipeline_options import PipelineOptions, StandardOptions
from apache_beam.io import ReadFromPubSub, WriteToBigQuery
from apache_beam.io.gcp.bigquery import BigQueryDisposition

# ─── CONFIGURATION ──────────────────────────────────────────────────
PROJECT_ID   = 'realtime-analytics-demo'
SUBSCRIPTION = 'projects/realtime-analytics-demo/subscriptions/realtime-events-
sub'
BIGQUERY_TABLE = 'realtime-analytics-demo:analytics_data.events'
# ────────────────────────────────────────────────────────────────────

# The BigQuery table schema matches what we created in Phase 2
TABLE_SCHEMA = {
    'fields': [
        {'name': 'event_id',   'type': 'STRING',    'mode': 'NULLABLE'},
        {'name': 'user_id',    'type': 'INTEGER',   'mode': 'NULLABLE'},
        {'name': 'event_type', 'type': 'STRING',    'mode': 'NULLABLE'},
        {'name': 'product',    'type': 'STRING',    'mode': 'NULLABLE'},
        {'name': 'amount',     'type': 'FLOAT',     'mode': 'NULLABLE'},
        {'name': 'timestamp',  'type': 'TIMESTAMP', 'mode': 'NULLABLE'},
        {'name': 'city',       'type': 'STRING',    'mode': 'NULLABLE'},
    ]
}

class ParseAndValidate(beam.DoFn):
    """Parses raw JSON bytes into a Python dictionary"""
    def process(self, element):
        try:
            # element is bytes, decode to string then parse JSON
            event = json.loads(element.decode('utf-8'))
            yield event          # Send to next step
        except Exception as e:
            logging.error(f'Failed to parse: {element}  Error: {e}')
            # Bad messages are just skipped (not sent to BigQuery)

def run():
    # Set pipeline options
    options = PipelineOptions()
    options.view_as(StandardOptions).streaming = True  # Real-time mode!

    logging.info('Starting pipeline...')
```

```
    with beam.Pipeline(options=options) as pipeline:
        (
            pipeline
            # Step 1: Read messages from Pub/Sub
            | 'Read from Pub/Sub' >> ReadFromPubSub(subscription=SUBSCRIPTION)

            # Step 2: Parse JSON bytes to dict
            | 'Parse JSON'        >> beam.ParDo(ParseAndValidate())

            # Step 3: Write each row to BigQuery
            | 'Write to BigQuery' >> WriteToBigQuery(
                table=BIGQUERY_TABLE,
                schema=TABLE_SCHEMA,
                write_disposition=BigQueryDisposition.WRITE_APPEND,  # Add rows
                create_disposition=BigQueryDisposition.CREATE_NEVER   # Table
exists
            )
        )

if __name__ == '__main__':
    logging.basicConfig(level=logging.INFO)
    run()
```

Step 6.2 — Check Your Subscription Name

Go to GCP Console → Pub/Sub → Topics → realtime-events → Subscriptions tab. You will see the subscription name. It should look like:

```
projects/realtime-analytics-demo/subscriptions/realtime-events-sub
```

Copy this exact name and paste it into SUBSCRIPTION in pipeline.py.

Phase 7: Run Everything Locally and Test

We will run on your laptop first (DirectRunner) to test. This is free and instant — no cloud processing costs!

Step 7.1 — Open TWO Terminal Windows

You need two separate terminal/command prompt windows open at the same time, both inside your realtime-analytics folder.

```
# In BOTH terminals, navigate to your project folder:
cd realtime-analytics
```

Step 7.2 — Start the Pipeline (Terminal 1)

In Terminal 1, run the Dataflow pipeline first:

```
# Terminal 1 — Start the pipeline
python pipeline.py

# You should see output like:
# INFO:root:Starting pipeline...
# INFO:apache_beam.runners.direct.direct_runner:Running pipeline...
# (It will appear to hang/wait — this is correct! It is waiting for messages)
```

The pipeline running and 'waiting' is correct behavior. It is listening for new messages from Pub/Sub.

Step 7.3 — Start the Data Producer (Terminal 2)

In Terminal 2, run the producer to start sending data:

```
# Terminal 2 — Start sending data
python producer.py

# You should see output like:
# Starting data producer...
# Publishing to: projects/realtime-analytics-demo/topics/realtime-events
# Press Ctrl+C to stop
# ----------------------------------------
# [1] Sent: page_view - Laptop - $0.0
# [2] Sent: purchase - Phone - $1243.50
# [3] Sent: click - Tablet - $0.0
# ... (new line every second)
```

Step 7.4 — Verify Data in BigQuery

After 30-60 seconds, check if data is appearing in BigQuery:

50. Go to GCP Console. Click on BigQuery in the left menu.
51. Expand: realtime-analytics-demo → analytics_data → events
52. Click on the events table.
53. Click 'QUERY TABLE' (top right).
54. This opens a SQL editor. Replace the SQL with:

```
-- Check how many rows we have
SELECT COUNT(*) as total_rows FROM `realtime-analytics-
demo.analytics_data.events`;

-- See the last 10 events
SELECT * FROM `realtime-analytics-demo.analytics_data.events`
ORDER BY timestamp DESC
LIMIT 10;

-- See total sales by product
SELECT
  product,
  COUNT(*) as total_events,
  SUM(amount) as total_revenue
FROM `realtime-analytics-demo.analytics_data.events`
WHERE event_type = 'purchase'
GROUP BY product
ORDER BY total_revenue DESC;
```

55. Click RUN. You should see results with rows from your producer!

If you see rows in BigQuery — CONGRATULATIONS! Your real-time pipeline is working. Data is flowing from your laptop to the cloud!

Phase 8: Build the Live Dashboard in Looker Studio

Looker Studio is completely FREE. It connects to BigQuery and shows beautiful live charts and graphs.

Step 8.1 — Open Looker Studio

56. Go to: lookerstudio.google.com
57. Sign in with the same Google account you used for GCP.
58. Click 'Create' → 'Report'.
59. A dialog will ask you to add a data source. Click 'BigQuery'.
60. Sign in again if asked. Then select:

- My Projects → realtime-analytics-demo
- Dataset → analytics_data
- Table → events

61. Click 'ADD'. Then click 'ADD TO REPORT' in the popup.

Step 8.2 — Add a Scorecard (Total Events)

62. In the toolbar at top, click 'Add a chart' → 'Scorecard'.
63. Draw a rectangle on your page.
64. In the right panel under 'Metric', choose Count of Records.
65. This shows the total number of events. Label it 'Total Events'.

Step 8.3 — Add a Time Series Chart (Events Over Time)

66. Click 'Add a chart' → 'Time series chart'.
67. Draw it on the page (make it wide).
68. Right panel: Dimension = timestamp, Metric = Count of Records.
69. This shows event frequency over time — a live pulse of your platform!

Step 8.4 — Add a Bar Chart (Revenue by Product)

70. Click 'Add a chart' → 'Bar chart'.
71. Draw it on the page.
72. Dimension = product, Metric = SUM of amount.
73. This shows which products are making the most money.

Step 8.5 — Add a Pie Chart (Event Types)

74. Click 'Add a chart' → 'Pie chart'.
75. Dimension = event_type, Metric = Count of Records.

76. Shows the distribution of page views vs clicks vs purchases.

Step 8.6 — Enable Auto-Refresh

77. Click anywhere on blank canvas (deselect all charts).
78. In the right panel, find 'Report settings' → 'Data freshness'.
79. Set it to 'Every 5 minutes' (minimum allowed by Looker Studio).

Your dashboard is live! Share it with your faculty by clicking 'Share' in the top right corner and copying the view link.

Phase 9: Run on Google Cloud (Full Cloud Deployment)

> OPTIONAL — This deploys the pipeline to actual Google Cloud servers. Only do this after local testing works. This will use some cloud credits.

Step 9.1 — Create a Cloud Storage Bucket (Required by Dataflow)

```
# Run this in your terminal
# Replace YOUR_BUCKET_NAME with something unique like: realtime-demo-bucket-
yourname

gsutil mb -p realtime-analytics-demo -l us-central1 gs://YOUR_BUCKET_NAME

# Example:
gsutil mb -p realtime-analytics-demo -l us-central1 gs://realtime-analytics-
yourname-2024
```

Step 9.2 — Modify pipeline.py for Cloud Runner

Change the run() function in pipeline.py to use Dataflow instead of local runner:

```
def run():
    # Cloud Dataflow options
    options = PipelineOptions([
        '--runner=DataflowRunner',
        '--project=realtime-analytics-demo',
        '--region=us-central1',
        '--temp_location=gs://YOUR_BUCKET_NAME/temp',
        '--staging_location=gs://YOUR_BUCKET_NAME/staging',
        '--job_name=realtime-events-pipeline',
        '--streaming',
    ])

    with beam.Pipeline(options=options) as pipeline:
        # ... rest of pipeline code stays the same ...
```

Step 9.3 — Deploy to Cloud

```
# This deploys the pipeline to Google Cloud Dataflow
python pipeline.py

# It will take 3-5 minutes to start
# You will see output like:
# INFO: Submitted job: 2024-xx-xx_xx_realtime-events-pipeline
# INFO: To cancel the job, run: gcloud dataflow jobs cancel ...
```

Step 9.4 — Monitor on GCP Console

80. In GCP Console, search for 'Dataflow'.
81. You will see your job listed with status 'Running'.
82. Click on it to see a visual graph of your pipeline stages.
83. You can see how many elements have been processed in real time!

Troubleshooting Common Errors

Error Message	What It Means	How to Fix
google.auth.exceptions.DefaultCredentialsError	Not logged into GCP	Run: gcloud auth application-default login
404 Resource not found: topic	Topic name is wrong	Check topic name in producer.py matches GCP console
403 Permission denied	API not enabled or wrong project	Enable the API in GCP Console and check project ID
ModuleNotFoundError: google.cloud	Library not installed	Run: pip install google-cloud-pubsub
BigQuery table not found	Wrong table path	Format must be: project:dataset.table
Pipeline hangs with no output	Normal behavior!	It is waiting for messages — run producer.py in second terminal

Final Project Folder Structure

```
realtime-analytics/
├── producer.py          # Sends fake data to Pub/Sub
├── pipeline.py          # Reads from Pub/Sub, writes to BigQuery
└── README.md            # (Optional) your notes

GCP Resources created:
├── Project: realtime-analytics-demo
├── Pub/Sub Topic: realtime-events
├── BigQuery Dataset: analytics_data
│   └── Table: events
├── Cloud Storage: gs://your-bucket-name  (for cloud deployment)
└── Looker Studio: your-dashboard-url
```

Summary — What You Built

Component	Technology	Status
Data Generator	Python (producer.py)	Sends 1 event/second
Message Queue	GCP Pub/Sub	Holds messages until processed
Stream Processor	Apache Beam / Dataflow	Parses & routes data
Data Warehouse	BigQuery	Stores all events in SQL table
Live Dashboard	Looker Studio	Real-time charts & graphs

You have built a production-grade real-time data analytics platform — the same architecture used by companies like Uber, Airbnb, and Spotify to process millions of events per second!

Good luck with your project!